

# 协程技术调研

从并发演进到底层机制、语言选型与实验验证

柯云超

计算机学院

并行程序设计课程展示

2026 年 6 月

- 1 问题：协程到底解决什么
- 2 机制：可暂停函数如何工作
- 3 语言：主流路线如何取舍
- 4 实验：本机结果说明什么
- 5 选型：什么时候该用协程

# 协程降低的是“等待”的代价， 不是“计算”的代价。

它把回调、阻塞线程和事件循环中的零散状态，还原成一段可以暂停、可以恢复的线性代码；代价没有消失，而是转移到编译器、运行时调度器和 I/O 后端。

### 本次汇报回答

- ❶ 为什么会从进程、线程、事件循环走到协程？
- ❷ 有栈、无栈协程在底层分别保存什么？
- ❸ Go、Java、C++、Python 等语言为什么选择不同路线？
- ❹ 本机实验说明协程擅长什么、不擅长什么？

# 并发模型演进：等待任务越来越多



## 本页结论

协程不是“更快的线程”，而是一种把大量阻塞等待从内核线程上挪开的组织方式。

## 普通函数

```
Resp handle(Req r) {  
    auto u = db.query(r.id);  
    auto d = rpc.fetch(u);  
    return render(d);  
}
```

调用栈一路向下，直到函数返回；如果 I/O 阻塞，承载它的线程也阻塞。

## 协程函数

```
Task<Resp> handle(Req r) {  
    auto u = co_await db.query(r.id);  
    auto d = co_await rpc.fetch(u);  
    co_return render(d);  
}
```

每个 `co_await` 都可能保存状态并交还执行权，之后由调度器恢复。



## Coroutine frame 保存什么

- 当前状态编号和恢复入口;
- 跨挂起点仍要使用的局部变量;
- promise / future / task 等返回协议对象;
- 异常、取消和最终挂起所需的元数据。

## 关键边界

无栈协程只能在被标记的 coroutine / async 函数内部挂起，因此会产生“函数颜色”问题。

# 有栈协程：保存的是调用栈

## 状态保存方式

- 每个任务有可保存的用户态栈；
- 挂起时保存栈指针、寄存器和调度状态；
- 恢复时把栈重新装载到运行线程上。

## 优势

可以在较深调用链中挂起，更接近“普通同步代码”的写法。

## 代价

运行时必须处理栈增长、栈复制、调度公平性和阻塞点识别。

goroutine / virtual thread

挂起时保存

可增长栈 / 可卸载栈帧

就绪时装载

少量 OS carrier 线程

内核调度

CPU 核心

## 三层协同：编译器 × 运行时 × OS



### 容易误解的一点

协程不是脱离操作系统的魔法。它必须依赖语言运行时识别“可等待事件”，再由 OS I/O 多路复用机制把“就绪”消息送回来。



# 主流语言：两条实现路线

| 语言      | 类型 | 调度器        | 一句话特点                                            |
|---------|----|------------|--------------------------------------------------|
| Go      | 有栈 | GMP 内建     | go + channel, 普通函数即可并发, 网络轮询和调度器一体化。             |
| Java 21 | 有栈 | JVM 调度     | 虚拟线程让同步代码低成本迁移, 但要注意 pinned carrier 等阻塞边界。       |
| C++20   | 无栈 | 标准不提供      | 语言只定义 co_await 协议, Task、executor、I/O 后端由库决定。     |
| Python  | 无栈 | 单线程事件循环    | async/await 上手快; 同步阻塞调用会卡住整个 loop, CPU 受 GIL 限制。 |
| C#      | 无栈 | 线程池 / 上下文  | async/await 工程化成熟, 异常传播和同步上下文规则需要理解。             |
| Rust    | 无栈 | Tokio 等运行时 | future 惰性执行、零成本抽象强, 但生命周期和 Pin 增加学习门槛。           |

## 本页结论

“有没有协程”不是最终答案; 调度器、标准库阻塞点覆盖和诊断工具决定它能不能在真实系统里用好。

# Go 与 Java：运行时接管阻塞点



Go GMP: G=goroutine, M=OS 线程, P= 运行队列。

## Go goroutine

运行时负责栈增长、work stealing、netpoller 和抢占；适合并发原生服务。

## Java virtual thread

JVM 在多数阻塞 I/O 处卸载虚拟线程栈帧，保留同步代码风格。

## 共同边界

阻塞点若不能被识别，或长期持有锁/本地调用，轻量任务仍可能压住 carrier。

## C++20 的低层特征

- `promise_type` 定义返回对象和挂起策略;
- `await_ready/suspend/resume` 定义等待协议;
- `coroutine_handle` 是恢复和销毁 frame 的句柄;
- 标准不规定 `executor`、`Task` 或异步 I/O。

## Python / Rust 的工程启示

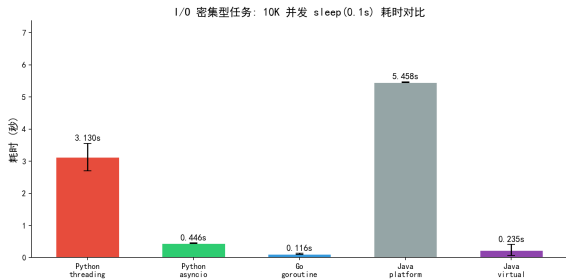
- Python `asyncio` 简洁, 但必须避免同步阻塞污染事件循环;
- Rust `future` 默认惰性, 必须由运行时 `poll` 推进;
- 无栈路线通常内存更省, 但 API 会明显区分同步与异步。

## 本页结论

无栈协程把“何时挂起、谁来恢复、frame 何时销毁”暴露得更清楚: 控制力更强, 工程约束也更多。

# I/O 密集型：协程快 7 到 23 倍

实验：10000 个逻辑任务，每个 sleep 100ms；5 次重复；平台为 i9-13900H + Windows 11。



## Python

threading 3.130s

asyncio 0.446s

约 7.0×

## Go

goroutine 0.116s

运行时轻量任务优势明显

## Java

platform 5.458s

virtual 0.235s

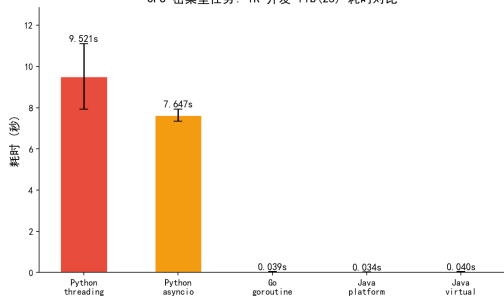
约 23.2×

## 本页结论

优势来自等待期间释放执行权，而不是把 sleep “算得更快”。

# CPU 密集与切换开销：协程不加速计算

CPU 密集型任务：1K 并发 fib(25) 耗时对比

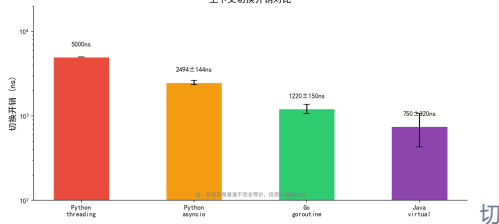


CPU 测试为 1000 个 fib(25) 任务；Go / Java 快主要来自多核并行。

## 解释边界

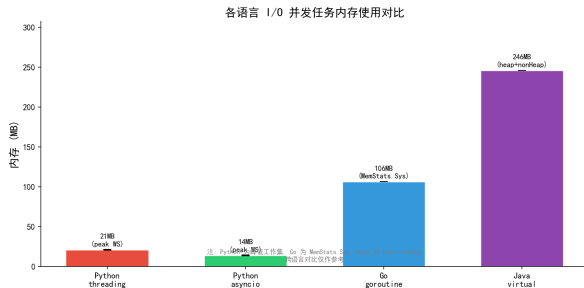
协程能减少等待占用的线程资源，但不能减少斐波那契递归本身的指令数；CPU 瓶颈仍应优先考虑并行计算、SIMD 或算法优化。

上下文切换开销对比



换测试并不同构：Python 为 1M 次 yield，Go/Java 为 100k 次 channel 或 sleep(0)。

# 内存结果：只能看趋势，不能简单排名



## 口径差异

- Python: 峰值进程 working set;
- Go: `runtime.MemStats.Sys`;
- Java: `heap + nonHeap`;
- 跨语言内存对比只用于数量级讨论。

## 实验局限

sleep 模拟 I/O, 不含真实网络协议栈; 切换微基准的操作语义不同。

## 常见失败模式

- 忘记取消子任务，形成协程泄漏；
- 超时只取消外层 future，底层 I/O 仍在跑；
- 同步阻塞函数进入事件循环；
- 错误在后台任务中被吞掉，调用方看不到失败。

## 更稳的实践

- 用作用域管理任务组和取消传播；
- 为每个等待点设置超时与资源释放路径；
- 对阻塞库做隔离：线程池、异步替代或运行时适配；
- 用 trace、dump、profile 观察协程堆积位置。

## 本页结论

协程的核心工程问题不是“创建任务有多快”，而是任务失败后谁负责收尾。

## ❶ 瓶颈是否真的在 I/O 等待？

如果瓶颈在 CPU，优先考虑多线程、多进程、SIMD、GPU 或算法优化。

## ❷ 阻塞 API 能否被运行时接管或替换？

Go / Java 21 更适合保留同步写法；C++ / Python / Rust 往往需要显式 async 化。

## ❸ 错误、取消、超时和背压能否结构化管理？

没有这层治理，协程越轻量，系统越容易积累“看不见的后台任务”。

## 本页结论

三问都能肯定回答时，协程通常比传统线程池更适合高并发 I/O。



## 结论 1：协程承载大规模 I/O 并发

它降低的是“等待”的代价，不是“计算”的代价。

## 结论 2：协程需要三层系统协同

编译器/解释器、语言运行时、OS I/O 多路复用共同完成挂起与恢复。

## 结论 3：选型要看调度器与生态

Go / Java 21 强调透明迁移；C++ / Python / C# / Rust 强调精细控制，各有场景。

## AI 使用说明

- 辅助梳理“演进-机制-语言-实验-选型”的展示结构；
- 辅助检查术语一致性、长句拆分和图表说明；
- 实验代码、图表和结论均以本机真实结果为准。

## 主要参考

Conway 1963; de Moura & Ierusalimsky 2009; JEP 444; PEP 492; C++20 coroutines; Go runtime; 同目录 bench.py、bench.go、Bench.java 与 results/。

# 谢谢!